



Insavlo Public API Documentation

Version: v1.2.7

Last Updated: 2026-06

Changes: Added webhook support.

Overview

Insavlo Public API provides document upload, processing, and result retrieval capabilities. It supports data extraction from various document formats including PDF, Word documents, and images.

API Features

- Single file and batch file upload support
- Asynchronous processing mode, suitable for large file processing
- Bearer Token-based authentication
- RESTful API design with JSON response format
- Webhook push support: receive results automatically when processing completes (no polling required)

Authentication

All API requests require Bearer Token authentication:

Authorization: Bearer YOUR_API_KEY

Getting Your API Key

Please contact our technical support team to obtain your dedicated API key.

Basic Information

Environment URLs

- **Demo Environment:** `https://demo.insavlo.com/insavlo/public-api` - For development testing and integration verification
- **Production Environment:**
 - Asia: `https://app-asia.insavlo.com/insavlo/public-api`
 - Europe: `https://app-eu.insavlo.com/insavlo/public-api`

Request Format

- **Content-Type:**
 - File upload: `multipart/form-data`
 - Status query: `application/json`
 - **Character Encoding:** UTF-8
-

API Endpoints

1. Upload Documents and Start Processing

Upload documents and create processing tasks in one step. Requires document files and corresponding skill code.

Endpoint: `POST /document_process/upload_and_process`

Request Format: `multipart/form-data`

Parameters:

Parameter	Type	Required	Description
files[]	File[]	Yes	Document files (single or multiple)
skill_code	String	Yes	Processing skill code
callback_url	String	No	Webhook URL to receive results when processing completes. The URL must first be registered in the Insavlo platform for the API credential, Must start with https:// (or

Parameter	Type	Required	Description
			http:// for internal environments). See Webhook Callback for details.

File Limitations:

- Supported formats: `.pdf`, `.jpg`, `.jpeg`, `.png`, `.doc`, `.docx`
- Single file size: Maximum 50MB
- Batch upload: Maximum 10 files, total size not exceeding 200MB

Single File Upload Example

Request (polling mode):

```
curl -X POST \  
  -H "Authorization: Bearer YOUR_API_KEY" \  
  -F "files[@invoice.pdf" \  
  -F "skill_code=invoice_v3" \  
https://demo.insavlo.com/insavlo/public-api/document_process/upload_and_process
```

Request (webhook push mode — add a registered `callback_url` to receive results automatically):

```
curl -X POST \  
  -H "Authorization: Bearer YOUR_API_KEY" \  
  -F "files[@invoice.pdf" \  
  -F "skill_code=invoice_v3" \  
  -F "callback_url=https://your-server.com/webhook/insavlo" \  
https://demo.insavlo.com/insavlo/public-api/document_process/upload_and_process
```

Success Response:

```
{
  "success": true,
  "transaction_id": "40b3ba73-3ecf-4bc1-8594-ed0216a72a33",
  "files_count": 1,
  "files": [
    {
      "file_id": "1e85997205ce4dbf96064bcf7abed782",
      "original_filename": "invoice.pdf"
    }
  ],
  "message": "1 file(s) uploaded and processing started successfully"
}
```

Batch File Upload Example

Request:

```
curl -X POST \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -F "files[]=@document1.pdf" \
  -F "files[]=@document2.jpg" \
  -F "files[]=@receipt.png" \
  -F "skill_code=invoice_v3" \
  https://demo.insavlo.com/insavlo/public-api/document_process/upload_and_process
```

Success Response:

```

{
  "success": true,
  "transaction_id": "40b3ba73-3ecf-4bc1-8594-ed0216a72a33",
  "files_count": 3,
  "files": [
    {
      "file_id": "1e85997205ce4dbf96064bcf7abed782",
      "original_filename": "document1.pdf"
    },
    {
      "file_id": "2f96008316df5ecf07074cdf8abed893",
      "original_filename": "document2.jpg"
    },
    {
      "file_id": "3a07119427ef6fdf18185def9bced904",
      "original_filename": "receipt.png"
    }
  ],
  "message": "3 file(s) uploaded and processing started successfully"
}

```

2. Query Processing Status

Query real-time status and results of document processing.

Endpoint: GET /document_process/status/{transaction_id}

Query Parameters:

Parameter	Type	Required	Default	Description
include_confidence_flag	Boolean	No	false	Whether to include confidence flag information in the response. Set to <code>true</code> to include confidence flags for extracted fields.

Request Example (without confidence):

```
curl -X GET \
  -H "Authorization: Bearer YOUR_API_KEY" \
  https://demo.insavlo.com/insavlo/public-api/document_process/status/40b3ba73-3ecf-4bc1-8594-e
```

Request Example (with confidence):

```
curl -X GET \
  -H "Authorization: Bearer YOUR_API_KEY" \
  https://demo.insavlo.com/insavlo/public-api/document_process/status/40b3ba73-3ecf-4bc1-8594-e
```

Response Example (without confidence flags):

```

{
  "success": true,
  "transaction_id": "40b3ba73-3ecf-4bc1-8594-ed0216a72a33",
  "status": "completed",
  "created_time": 1704067200000,
  "updated_time": 1704067260000,
  "files": [
    {
      "file_id": "1e85997205ce4dbf96064bcf7abed782",
      "file_name": "invoice.pdf",
      "status": "completed",
      "page_count": 2,
      "file_size": 1024000,
      "skill_code": "invoice_v3",
      "result": {
        "INVOICE_NO": "INV-2024-001",
        "INVOICE_DATE": "2024-01-01",
        "INVOICE_TOTAL_AMOUNT": "1,026.50",
        "BILL_CURRENCY": "SGD",
        "BENEFICIARY_NAME": "GLOBAL AIRFREIGHT INTERNATIONAL PTE LTD",
        "ITEMS": [
          {
            "NO.": "1",
            "DESCRIPTION": "WIDE TEK 36 WIDE FORMAT SCANNER",
            "QTY": "1 PCS",
            "AMOUNT": "1,026.50"
          }
        ]
      }
    }
  ]
}

```

Response Example (with confidence flags - include_confidence_flag=true):

When `include_confidence_flag=true`, each field in the result will include a `$value` and `$confidence_flag`:

- `$value`: The extracted field value

- `$confidence_flag` : Confidence indicator ("1" = high confidence, "0" = low confidence, may need manual verification)


```
{
  "success": true,
  "transaction_id": "40b3ba73-3ecf-4bc1-8594-ed0216a72a33",
  "status": "completed",
  "created_time": 1704067200000,
  "updated_time": 1704067260000,
  "files": [
    {
      "file_id": "1e85997205ce4dbf96064bcf7abed782",
      "file_name": "invoice.pdf",
      "status": "completed",
      "page_count": 2,
      "file_size": 1024000,
      "skill_code": "invoice_v3",
      "result": {
        "BILL_CURRENCY": {
          "$value": "CNY",
          "$confidence_flag": "1"
        },
        "INVOICE_DATE": {
          "$value": "2023-04-05",
          "$confidence_flag": "1"
        },
        "INVOICE_TOTAL_AMOUNT": {
          "$value": "1415.00",
          "$confidence_flag": "1"
        },
        "INVOICE_DUE_DATE": {
          "$value": "",
          "$confidence_flag": "1"
        },
        "INVOICE_NO": {
          "$value": "25543855",
          "$confidence_flag": "1"
        },
        "BENEFICIARY_NAME": {
          "$value": "TEST BENEFICARY NAME",
          "$confidence_flag": "0"
        }
      }
    }
  ]
}
```

```
"ITEMS": [  
  {  
    "NO.": {  
      "$value": "1",  
      "$confidence_flag": "1"  
    },  
    "UNIT_PRICE": {  
      "$value": "1334.91",  
      "$confidence_flag": "1"  
    },  
    "QTY": {  
      "$value": "1",  
      "$confidence_flag": "1"  
    },  
    "AMOUNT": {  
      "$value": "1334.91",  
      "$confidence_flag": "1"  
    },  
    "TAX": {  
      "$value": "80.09",  
      "$confidence_flag": "1"  
    },  
    "DESCRIPTION": {  
      "$value": "Food Service",  
      "$confidence_flag": "1"  
    },  
    //...more fields skipped  
  }  
,  
  "TOTAL_TAX": {  
    "$value": "80.09",  
    "$confidence_flag": "1"  
  },  
  "SUBTOTAL_WITHOUT_TAX": {  
    "$value": "1334.91",  
    "$confidence_flag": "1"  
  },  
  //...more fields skipped  
}  
}
```

```
]
}
```

3. Webhook Callback (Push)

Instead of polling, you can provide a `callback_url` when submitting a task. The `callback_url` must first be registered in the Insavlo platform for the API Key that you use to call this API. When all files in a transaction finish processing, Insavlo will **HTTP POST** the full result to your URL automatically.

Before using webhook push:

1. Enable Webhook for the API credential in the Insavlo platform.
2. Register one or more allowed callback URLs for that credential and obtain the Webhook Secret provided by Insavlo.
3. Use one of those registered URLs as the `callback_url` parameter in `upload_and_process`.
4. Use the Webhook Secret provided by Insavlo to verify the request signature.

Trigger condition: The push fires once, after the transaction status transitions to `completed` (i.e., all files reach `completed` or `error`).

Retry policy: If your endpoint is unreachable or returns a non-2xx response, Insavlo retries with exponential back-off:

Attempt	Delay before retry
1st retry	30 s
2nd retry	60 s
3rd retry	120 s
...	doubles each time
Max retries	8 (after which the task is marked <code>failed</code>)

Webhook Request

Insavlo sends a `POST` request to your `callback_url` with the following headers:

Header	Value	Description
Content-Type	application/json; charset=utf-8	
X-Webhook-Event	document_process.completed	Event type
X-Transaction-Id	<transaction_id>	The transaction that completed
X-Webhook-Signature	sha256=<hex>	HMAC-SHA256 signature computed with the Webhook Secret provided by Insavlo

Webhook Payload

The body mirrors the `GET /status` response:

```
{
  "event": "document_process.completed",
  "transaction_id": "40b3ba73-3ecf-4bc1-8594-ed0216a72a33",
  "status": "completed",
  "skill_code": "invoice_v3",
  "files": [
    {
      "file_id": "1e85997205ce4dbf96064bcf7abed782",
      "file_name": "invoice.pdf",
      "status": "completed",
      "skill_code": "invoice_v3",
      "result": {
        "INVOICE_NO": "INV-2024-001",
        "INVOICE_DATE": "2024-01-01",
        "INVOICE_TOTAL_AMOUNT": "1,026.50",
        "BILL_CURRENCY": "SGD"
      }
    }
  ],
  "timestamp": "2026-06-14T12:00:00Z"
}
```

Your endpoint must respond with HTTP 2xx within 15 seconds, otherwise Insavlo treats the attempt as failed and schedules a retry.

Webhook Secret & Signature Verification

To protect against spoofed requests, verify every push with the **Webhook Secret** provided by Insavlo for your API credential. Insavlo includes the following header in each webhook request:

```
X-Webhook-Signature: sha256=<hmac-sha256-hex>
```

The signature is computed as `HMAC-SHA256(webhook_secret, raw_request_body)`.

Verification example (Python):

```
import hmac
import hashlib

def verify_webhook(request_body: bytes, signature_header: str, secret: str) -> bool:
    """
    Returns True if the request genuinely came from Insavlo.
    Call this before processing any webhook payload.
    """
    expected = 'sha256=' + hmac.new(
        secret.encode('utf-8'),
        request_body,
        hashlib.sha256,
    ).hexdigest()
    return hmac.compare_digest(expected, signature_header)
```

Security note: Always use `hmac.compare_digest` (constant-time comparison) to prevent timing attacks. Treat a missing or invalid `X-Webhook-Signature` header as an authentication failure.

Status Descriptions

Transaction Status

Status	Description
<code>in_progress</code>	Processing in progress (some files may be completed while others are still being processed, awaiting verification, or exporting)

Status	Description
completed	All files have been processed (check each file's status and msg field to confirm success or failure)
error	Error occurred during transaction processing

Important Note: Even when the transaction status is `in_progress` , some files may have already completed processing. It's recommended to check each file's status for accurate information.

File Status

Status	Description
queued	Queued for processing
in_progress	Processing in progress
pending_verification	Awaiting manual verification (AI processing completed, awaiting human confirmation)
exporting	Exporting to third-party system
completed	Processing completed (check msg field to confirm success)
error	Processing failed (check msg field for failure reason)

Skill Codes

A skill specializes in extracting a specific type of document and provides specified structured data.

Here are some basic examples:

Skill Code	Description
invoice_v3	Invoice data extraction
receipt_v2	Receipt data extraction

For more skills, please contact the Insavlo team. We customize skills based on actual needs and provide corresponding skill codes.

Error Codes

HTTP Status Code	Error Type	Description
200	Success	Request processed successfully
400	Bad Request	Parameter format error, unsupported file type, etc.
401	Authentication Failed	Invalid or missing API key
403	Permission Denied	Insufficient skill code permissions
404	Resource Not Found	Transaction ID does not exist
413	File Too Large	Exceeds file size limit
429	Rate Limit Exceeded	Too many requests
500	Server Error	Internal processing error

Error Response Format

```
{
  "success": false,
  "error": "Error description message"
}
```

Integration Guide

Choosing a Workflow

Insavlo supports two integration patterns. Choose based on your architecture:

	Polling	Webhook Push
How it works	Your code calls <code>GET /status</code> repeatedly	Insavlo calls your endpoint when done
Best for	Simple scripts, batch jobs, synchronous flows	Web services, event-driven systems

	Polling	Webhook Push
Requirements	None	A publicly reachable HTTPS endpoint registered in Insavlo
Setup effort	Minimal	Enable Webhook and register allowed callback URLs in Insavlo

Workflow A — Polling

1. Upload and Start Processing

```
POST /document_process/upload_and_process
```

Get `transaction_id`

2. Poll Status

```
GET /document_process/status/{transaction_id}
```

Recommended to query every 10 seconds until status is `completed` or `error`

3. Get Results

When status is `completed`, retrieve extracted data from the `files[].result` field in the response

Workflow B — Webhook Push

1. Enable Webhook in Insavlo

Register your callback URL for the API Key and copy the Webhook Secret provided by Insavlo.

2. Upload and Start Processing — include the registered `callback_url`

```
POST /document_process/upload_and_process
-F callback_url=https://your-server.com/webhook/insavlo
```

Get `transaction_id`

3. Wait for Insavlo to call your endpoint

When all files finish processing, Insavlo POSTs the complete result to `callback_url`. No polling needed.

4. **Verify and handle the payload**

Validate the `x-Webhook-Signature` header using the Webhook Secret provided by Insavlo, then process `files[].result`.

Python Integration Examples

Polling Client

```
import requests
import time
import json

class InsavloClient:
    def __init__(self, base_url, api_key):
        self.base_url = base_url
        self.headers = {"Authorization": f"Bearer {api_key}"}

    def process_document(self, file_path, skill_code, callback_url=None):
        """Upload a document and start processing.

        Args:
            file_path: Path to the document file.
            skill_code: Processing skill code.
            callback_url: Optional registered webhook URL. When provided, Insavlo
                        will POST the result to this URL when processing completes.
        """
        url = f"{self.base_url}/document_process/upload_and_process"
        with open(file_path, 'rb') as f:
            data = {'skill_code': skill_code}
            if callback_url:
                data['callback_url'] = callback_url
            response = requests.post(
                url, headers=self.headers,
                files={'files[]': f}, data=data,
            )
            response.raise_for_status()
            return response.json()

    def get_status(self, transaction_id, include_confidence_flag=False):
        """Query processing status."""
        url = f"{self.base_url}/document_process/status/{transaction_id}"
        params = {}
        if include_confidence_flag:
            params['include_confidence_flag'] = 'true'
```

```
response = requests.get(url, headers=self.headers, params=params)
return response.json()
```

```
def wait_for_completion(self, transaction_id, timeout=300, include_confidence_flag=False):
    """Poll until all files in a transaction reach a terminal state.

    Note: The transaction status may remain 'in_progress' while individual files
    are still processing. Check each file's status – processing is complete only
    when all files are 'completed' or 'error'.
    """
    start_time = time.time()
    while time.time() - start_time < timeout:
        result = self.get_status(transaction_id, include_confidence_flag=include_confidence)
        transaction_status = result.get('status')
        print(f'Transaction Status: {transaction_status}')

        files = result.get('files', [])
        if files:
            all_done = all(f.get('status') in ('completed', 'error') for f in files)
            for f in files:
                print(f'File {f.get("file_name")}: {f.get("status")}')
            if all_done:
                return result

        if transaction_status == 'error':
            raise Exception(f"Processing failed: {result}")

        time.sleep(10)

    raise TimeoutError("Processing timeout")
```

— Usage: polling mode —————

```
client = InsavloClient(
    base_url="https://demo.insavlo.com/insavlo/public-api",
    api_key="YOUR_API_KEY",
)

result = client.process_document("invoice.pdf", "invoice_v3")
```

```
transaction_id = result['transaction_id']

final_result = client.wait_for_completion(transaction_id)
extracted_data = final_result['files'][0]['result']
print(json.dumps(extracted_data, indent=2, ensure_ascii=False))

# — Usage: webhook mode (no polling needed) —————
# Before using this URL, register it for the API credential in the Insavlo platform.

result = client.process_document(
    "invoice.pdf", "invoice_v3",
    callback_url="https://your-server.com/webhook/insavlo",
)
print(f"Submitted. Transaction ID: {result['transaction_id']}")
print("Result will be pushed to your callback_url when processing completes.")
```

Webhook Receiver (Flask)

```
import hmac
import hashlib
import json
from flask import Flask, request, jsonify

app = Flask(__name__)

WEBHOOK_SECRET = "your_insavlo_webhook_secret_here"

def verify_signature(body: bytes, header: str) -> bool:
    expected = 'sha256=' + hmac.new(
        WEBHOOK_SECRET.encode('utf-8'), body, hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(expected, header or '')

@app.route('/webhook/insavlo', methods=['POST'])
def receive_webhook():
    body = request.get_data()
    sig = request.headers.get('X-Webhook-Signature', '')

    if not verify_signature(body, sig):
        return jsonify({'error': 'Invalid signature'}), 401

    event = json.loads(body)
    print(f"Received event: {event['event']} for transaction {event['transaction_id']}")

    for file in event.get('files', []):
        file_name = file['file_name']
        file_status = file['status']
        print(f" {file_name}: {file_status}")

        if file_status == 'completed':
            result = file['result']
            # process extracted data ...
            pass
        elif file_status == 'error':
```

```
print(f" => Processing failed for {file_name}")
```

```
# Must return 2xx within 15 seconds to prevent retry
```

```
return jsonify({'received': True}), 200
```

```
if __name__ == '__main__':
```

```
    app.run(port=8080)
```